

IMPERMAX · SECURITY REVIEW

Impermax Tokenized Aerodrome Position.

A security review of Tokenized Aerodrome Position.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Impermax	Tokenized Aerodrome Position	Guardian	July 30 – August 4, 2025	August 12, 2025

§ 00

Contents.

20 pages — overview, confidence ranking, scope, 11 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 11 findings	P. 08
§ 05	Disclaimer	P. 19
§ 06	About Guardian	P. 20

§ 01

Overview.

Guardian reviewed Tokenized Aerodrome Position across a single review round. The final report includes 1 High, 2 Medium, 6 Low, 2 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Impermax-Finance/impermax-v3-core
CLIENT	METHODOLOGY
Impermax	Smart contract · Manual · Main Review
PROJECT	
Tokenized Aerodrome Position	
PREPARED BY	
Guardian	
AUDIT WINDOW	
July 30 – August 4, 2025	
FINAL REPORT	
August 12, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	0	0	0	0	0
● High	1	0	0	0	1
● Medium	2	0	1	0	1
● Low	6	0	6	0	0
● Informational	2	0	1	0	1
All severities	11	0	8	0	3

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Tokenized Aerodrome Position in Impermax-Finance/impermax-v3-core . The scope includes 187 source lines across 1 file.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/extensions/TokenizedAeroCLPosition.sol	187	257

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	ETH Refund Can Revert Critical Functions	DoS	HIGH	RESOLVED	p. 8
M-01	Full Split Causes Revert	Logical Error	MEDIUM	RESOLVED	p. 9
M-02	getPositionData Will Revert For Certain Prices	DoS	MEDIUM	ACKNOWLEDGED	p. 10
L-01	Unclaimed Fees Are Lost After mint	Logical Error	LOW	ACKNOWLEDGED	p. 11
L-02	Lack Of Slippage Protection	MEV	LOW	ACKNOWLEDGED	p. 12
L-03	ecrecover Allows Signature Malleability	Signatures	LOW	ACKNOWLEDGED	p. 13
L-04	Unclaimed Dust After split	Warning	LOW	ACKNOWLEDGED	p. 14
L-05	Rewards Cannot Be Claimed By EOA	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 15
L-06	Unused Tokens Not Returned	Logical Error	LOW	ACKNOWLEDGED	p. 16
I-01	Naming Convention For _addGauge	Informational	INFORMATIONAL	RESOLVED	p. 17
I-02	Gas Optimization For nonReentrant Modifier	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 18

HIGH **H-01**

RESOLVED

ETH Refund Can Revert Critical Functions

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team2/blob/9d9561e104f8aea60373bcdab46b6582f618a81e/contracts/extensions/TokenizedAeroCLPosition.sol#L167
REVIEW	Main Review

DESCRIPTION

The `increaseLiquidity` function in the `TokenizedAeroCLPosition` contract allows users to add liquidity to an existing position. It does so by withdrawing the user's position from the gauge, calling `increaseLiquidity` on the `NonfungiblePositionManager` with the original and additional amounts, and then redepositing the position.

However, the `NonfungiblePositionManager`'s `increaseLiquidity` function ends by calling `refundETH`, which sends any residual ETH in the contract back to `msg.sender`. In this context, `msg.sender` is the `TokenizedAeroCLPosition` contract itself. Since the contract lacks a `fallback` function, the refund fails and causes the entire transaction to revert.

A malicious actor could exploit this by sending a small amount of ETH to the `NonfungiblePositionManager` contract before a `increaseLiquidity` call, ensuring that the `refundETH` call fails and reverts the transaction.

The same issue exists in the `split` function, which also calls `mint` on the `NonfungiblePositionManager`, triggering a similar refund. This is more critical since `split` is used in liquidation flows—meaning an attacker could block or delay liquidations by intentionally triggering a refund failure.

RECOMMENDATION

Consider adding a `fallback` function to ensure the contract can safely receive ETH refunds and avoid unexpected reverts.

RESOLUTION

Status: Resolved.

<https://github.com/Impermax-Finance/impermax-v3-core/commit/1f6e4b37cb932221ee72ff661fab35252d62232>

MEDIUM **M-01**

RESOLVED

Full Split Causes Revert

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team2/blob/9d9561e104f8aea60373bcdab46b6582f618a81e/contracts/extensions/TokenizedAeroCLPosition.sol#L167
REVIEW	Main Review

DESCRIPTION

The `split` function allows users to split their position by a specified percentage, with 100% being the maximum. However, if a user attempts a full (100%) split, the `decreaseAndMint` function in the `NfpmAeroInteractions` library burns the original `tokenId`, since no liquidity remains in the original position.

The issue arises when the `split` function subsequently attempts to redeposit the now-burned `tokenId` into the gauge. This causes a revert, making a full split impossible. Additionally, because the original token is burned without claiming any pending fees from the gauge, those rewards are lost for the user.

A similar issue occurs when a user passes in 0% to `split` — the function attempts to mint a new position with no liquidity, which results in the new NFT never being minted.

RECOMMENDATION

Prevent 0% and 100% splits by modifying the logic to require `percentage > 0 && percentage < 1e18`, or alternatively, add checks to ensure fees are claimed and that the original token is not redeposited after being burned.

RESOLUTION

Status: Resolved.

<https://github.com/Impermax-Finance/impermax-v3-core/commit/9d9a4ee9f7490fc53b466e9576511421ca643962>

MEDIUM **M-02**

ACKNOWLEDGED

getPositionData Will Revert For Certain Prices

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team2/blob/9d9561e104f8aea60373bcdab46b6582f618a81e/contracts/extensions/TokenizedAeroCLPosition.sol#L108
REVIEW	Main Review

DESCRIPTION

The `getPositionData` function is intended to return price and liquidity information for a given position. It computes values such as `currentPrice`, `lowestPrice`, and `highestPrice` using the current price and a user-defined safety margin. These values are constrained using the `safe160` helper to ensure they fit within a `uint160`.

However, in certain token pairs where the price can near the maximum limit representable in Uniswap V3, the computed `highestPrice` can overflow the `uint160` range. When this occurs, the `safe160` check will revert the transaction, even though the position itself remains valid within Uniswap.

This is particularly problematic because `getPositionData` is used in several critical functions, including liquidation checks. A revert in this context could block or delay important protocol operations.

RECOMMENDATION

Ensure newly added token pairs do not result in price ranges exceeding the `uint160` max, as this can disrupt core functions.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Unclaimed Fees Are Lost After mint

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team1/blob/main/contracts/extensions/TokenizedAeroCLPosition.sol#L133
REVIEW	Main Review

DESCRIPTION

When `mint` is called, it deposits into the gauge, which triggers a collection of any accrued LP fees. These fees are transferred to `TokenizedAeroCLPosition`. However, unless the caller explicitly invokes `skim`, these tokens remain unclaimed and can be taken by anyone. As a result, the user who called `mint` may unintentionally forfeit their accrued LP fees if they do not immediately follow up with a `skim`.

RECOMMENDATION

Consider calling `skim(msg.sender)` at the end of the `mint` function.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Lack Of Slippage Protection

CATEGORY	MEV
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team2/blob/9d9561e104f8aea60373bcdab46b6582f618a81e/contracts/extensions/libraries/NfpmAeroInteractions.sol#L49
REVIEW	Main Review

DESCRIPTION

The `increaseLiquidity` function enables users to add liquidity to their existing position. However, both `amount0Min` and `amount1Min` are hardcoded to zero, meaning no slippage protection is applied during the minting process. This exposes users to unfavorable price movements or MEV attacks.

The same issue exists in the `split` function, which reduces a position by a specified percentage and mints a new position with the withdrawn liquidity. Here too, `amount0Min` and `amount1Min` are set to zero, exposing users to similar risks.

RECOMMENDATION

Consider adding slippage protection by allowing user-defined `amount0Min` and `amount1Min` values or by setting reasonable minimum thresholds to reduce the risk of poor execution or MEV exploitation.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

ecrecover Allows Signature Malleability

CATEGORY	Signatures
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team1/blob/main/contracts/ImpermaxERC721.sol#L160
REVIEW	Main Review

DESCRIPTION

`ImpermaxERC721.sol` uses vanilla `ecrecover` for signer recovery, which is susceptible to malleability due to signature variations.

This does not cause any immediate damage since the signed permit itself does not change. However, this should still be considered for improvement.

RECOMMENDATION

Consider using OpenZeppelin's ECDSA library for signer recovery to mitigate malleability risks.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Unclaimed Dust After split

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team1/blob/main/contracts/extensions/TokenizedAeroCLPosition.sol#L167
REVIEW	Main Review

DESCRIPTION

`split` removes some liquidity from the current LP position and mints a new one. Due to Aero rounding during minting, small residual (dust) token amounts can remain in the `TokenizedAeroCLPosition` contract. These residual tokens left behind can be skimmed by anyone, If not reclaimed, the owner of the original position loses these tokens.

RECOMMENDATION

Considering calling `skim` to the position owner after `split`.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Rewards Cannot Be Claimed By EOA

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team1/blob/main/contracts/extensions/TokenizedAeroCLPosition.sol#L215
REVIEW	Main Review

DESCRIPTION

In the `claim` function, `_checkAuthorizedCollateral` obtains owner of the `tokenId` from the Collateral contract:

```
address collateral = _requireOwned(tokenId);  
address owner = IERC721(collateral).ownerOf(tokenId);
```

This assumes that every user holding the wrapper NFT will deposit into the Collateral contract. If the wrapper NFT is still in a EOA wallet or separate contract, then the `ownerOf` call will revert. This blocks anyone from calling `claim` when the wrapper NFT is not being used as collateral.

RECOMMENDATION

Consider if this is expected behavior and document this risk for users.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Unused Tokens Not Returned

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team2/blob/9d9561e104f8aea60373bcdab46b6582f618a81e/contracts/extensions/TokenizedAeroCLPosition.sol#L190
REVIEW	Main Review

DESCRIPTION

The `increaseLiquidity` function in the `TokenizedAeroCLPosition` contract allows users to add liquidity to an existing position. The user must first transfer `token0` and `token1` to the contract, then call `increaseLiquidity`. The function withdraws the user's position from the gauge, attempts to increase its liquidity using the transferred amounts, and then redeposits the position.

However, if the provided token amounts are unbalanced, the actual liquidity added may use only part of the transferred tokens. Any remaining tokens stay in the contract without being returned to the user. These leftover tokens can later be claimed by anyone through the `skim` function, resulting in a potential loss of funds for the user.

RECOMMENDATION

Consider modifying `increaseLiquidity` to automatically return unused tokens to the user. Alternatively, clearly document this behaviour and ensure users are advised to call `skim` immediately after `increaseLiquidity` to recover any remaining tokens.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

RESOLVED

Naming Convention For `_addGauge`

CATEGORY Informational**LOCATION** <https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team1/blob/main/contracts/extensions/TokenizedAeroCLPosition.sol#L232>**REVIEW** Main Review

DESCRIPTION

The function `_addGauge` is declared `external` but is named with a leading underscore, a convention usually reserved for `private` or `internal` functions.

RECOMMENDATION

Rename `_addGauge` to `addGauge` to align with standard Solidity conventions.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

ACKNOWLEDGED

Gas Optimization For nonReentrant Modifier

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/impermax-v3-core-impermaxtokenizedaerodromeposition-team1/blob/main/contracts/extensions/TokenizedAeroCLPosition.sol#L262>

REVIEW Main Review

DESCRIPTION

The `nonReentrant` modifier in `TokenizedAeroCLPosition.sol` currently relies on a `bool` flag (`_notEntered`) to prevent reentrancy. However, this approach incurs high gas costs because each call involves a storage write from zero to nonzero (and vice versa).

RECOMMENDATION

Update the `nonReentrant` modifier to use a `uint256` two-state pattern (as recommended by OpenZeppelin), which avoids zero-value writes and reduces gas usage by approximately 10,000–15,000 per call.

```
// storage
uint256 private constant _NOT_ENTERED = 1;
uint256 private constant _ENTERED = 2;
uint256 private _status;
// in your constructor or initialize:
_status = _NOT_ENTERED;
// modifier
modifier nonReentrant() {
    require(_status != _ENTERED, "Impermax: REENTERED");
    _status = _ENTERED;
    _;
    _status = _NOT_ENTERED;
}
```

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.